

INFORME DE ARQUITECTURA & CIBERSEGURIDAD

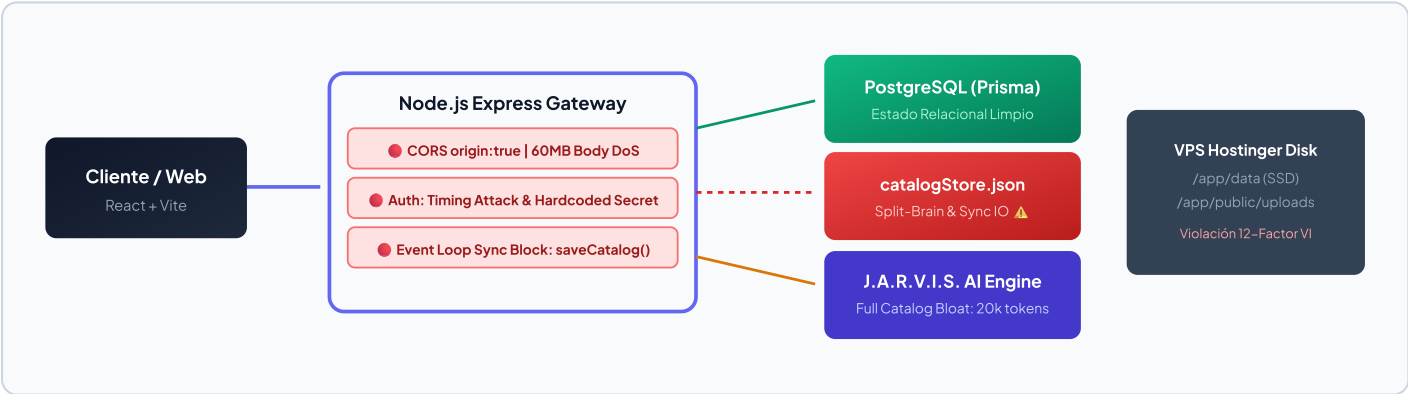
Radiografía Estructural, Vulnerabilidades OWASP y Plan de Madurez Empresarial

PROYECTO	EVALUADOR	ENTORNO	FECHA
Deco Vintage Guate (Deko Labs)	Staff Architect & AppSec Lead	Producción (decovintage.online)	31 de Agosto, 2026

1. Resumen Ejecutivo (Executive Summary)

Estado Operativo: El proyecto ha alcanzado una etapa de **estabilidad operativa básica** tras superar el hito crítico de compatibilidad en librerías nativas C++ (glibc 2.36 y libvips42 en Docker), estructurar la persistencia relacional con PostgreSQL vía Prisma ORM y estabilizar el fallback de la Single Page Application (SPA).

Diagnóstico Arquitectónico: Sin embargo, el sistema opera actualmente bajo una **Arquitectura de Transición con Cerebro Dividido (Split-Brain)**. Coexisten fallbacks inseguros de credenciales maestras, comparaciones criptográficas vulnerables a *Side-Channel Timing Attacks*, operaciones de I/O síncronas bloqueantes en el Event Loop de Node.js, exposición DoS por límites desproporcionados de JSON (60MB) y almacenamiento de medios acoplado al disco local, lo que viola el Factor VI de *The 12-Factor App* e impide el escalado horizontal.



2. Fortalezas Arquitectónicas Identificadas

1. Aislamiento Multi-Etapa en Docker con Paridad de Toolchains (glibc / OpenSSL)

El uso unificado de `node:20-bookworm-slim` en ambas etapas (`build` y `production`) garantiza que los binarios nativos C++ compilados por `sharp` y el motor de consultas de Prisma compartan exactamente `glibc 2.36` y `openssl 3.0.x`, erradicando fallos fatales de `dlopen()` en el inicio del contenedor.

2. Modelado Relacional Tipado e Integridad Referencial en Prisma

El esquema en `prisma/schema.prisma` implementa tipos `Decimal(10,2)` para evitar distorsiones de punto flotante de JS en moneda, eliminación en cascada (`onDelete: Cascade` en `PosterSize`) e índices compuestos (`@@unique([posterId, sizeId])`) que previenen registros huérfanos.

3. Failover Resiliente en Cascada del Motor de IA (J.A.R.V.I.S.)

La arquitectura en `services/jarvisService.js` implementa una degradación ordenada: primero consulta el SDK moderno `@google/genai` con rotación de claves/modelos, conmuta a Google Cloud Vertex AI REST y finalmente cae a un motor heurístico local offline, garantizando disponibilidad continua del chat.

4. Capa Anti-Corrupción / Patrón Adaptador en Dominio

La función `formatPosterForClient` en `catalogService.js` desacopla la normalización de la base de datos (con nombres canónicos en español) respecto a las propiedades consumidas por los componentes React (en inglés), permitiendo refactorizaciones sin alterar contratos del frontend.

5. Estrategia de Entrega y Caché HTTP Diferenciada

En `server.js` se configuran cabeceras inmutables (`maxAge: 1y`) para assets estáticos con hash y desactivación estricta de caché (`no-cache, no-store`) para el archivo `index.html` de la SPA, evitando que los usuarios finales queden atrapados en versiones obsoletas tras un despliegue.

3. 🚨 Deuda Técnica Crítica (Nivel Rojo — Riesgo de Intrusión o Quiebre)

● CRIT-01: Credenciales y Claves Maestras Hardcodeadas con Fallback Inseguro

CWE-798 / OWASP A07

Ubicación: `middleware/auth.js:10-12` | Severidad: CRÍTICA (CVSS 9.1)

```
// CÓDIGO VULNERABLE ACTUAL:
const ADMIN_USER = process.env.ADMIN_USER || 'SebasDmente';
const ADMIN_PASS = process.env.ADMIN_PASSWORD || '4214294880101';
const AUTH_SECRET = process.env.ADMIN_SECRET || 'deco_vintage_guate_secret_2026_master_key';
```

🌟 **Vector de Explotación & Impacto:** Si un despliegue en Dokploy omite las variables de entorno o la configuración se corrompe en el VPS, el servidor adopta silenciosamente credenciales conocidas públicamente en el repositorio. Un atacante puede generar firmas HMAC válidas y tomar control administrativo completo de la plataforma.

💡 **Remediación Mandatoria (Fail-Fast):** Abortar el proceso al arrancar (`process.exit(1)`) si las variables requeridas no están definidas en el entorno. Prohibir totalmente fallbacks por defecto en código fuente.

● CRIT-02: Vulnerabilidad a Timing Attacks en Verificación HMAC

CWE-208 / OWASP A02

Ubicación: `middleware/auth.js:44` | Severidad: ALTA (CVSS 7.4)

```
// CÓDIGO VULNERABLE:
if (sig !== expectedSig) return false;

// REMEDIACIÓN CRIPTOGRÁFICA EN TIEMPO CONSTANTE:
const sigBuf = Buffer.from(sig, 'hex');
const expBuf = Buffer.from(expectedSig, 'hex');
if (sigBuf.length !== expBuf.length || !crypto.timingSafeEqual(sigBuf, expBuf)) return false;
```

🌟 **Vector de Explotación:** La comparación estándar `!==` en el motor V8 se detiene en el primer byte diferente. Mediante análisis estadístico de microsegundos de respuesta HTTP (*Side-Channel Timing Attack*), un atacante puede deducir la firma HMAC carácter por carácter.

● CRIT-03: Persistencia Dual ("Split-Brain") y Bloqueos Síncronos del Event Loop

ARQUITECTURA ANTIPATRÓN / DOS

Ubicación: `services/catalogService.js:454-478` | Severidad: ALTA (Disponibilidad)

```
// CÓDIGO BLOQUEANTE EN EL EVENT LOOP:
export function saveCatalog(dataObject) {
  fs.writeFileSync(tmpFile, JSON.stringify(dataObject, null, 2), 'utf-8');
  fs.renameSync(tmpFile, CATALOG_FILE);
}
```

🌟 **Impacto en Rendimiento:** Coexisten dos fuentes de verdad (PostgreSQL y `catalogStore.json`). Las llamadas síncronas `fs.writeFileSync` y `fs.renameSync` congelan el Event Loop en el hilo principal de Node.js. Peticiones concurrentes de clientes (consultas de catálogo, chats con J.A.R.V.I.S., pagos) quedan encoladas provocando picos de latencia de varios segundos y errores 504 Gateway Timeout.

● CRIT-04: CORS Universal con Credentials y Límite Global de 60 MB (Riesgo DoS por OOM)

OWASP A05:2021 / CWE-400

Ubicación: `server.js:51-55` | Severidad: ALTA (CVSS 7.5)

```
app.use(cors({ origin: true, credentials: true }));
app.use(express.json({ limit: '60mb' }));
app.use(express.urlencoded({ extended: true, limit: '60mb' }));
```

✦ **Impacto:** `origin: true` refleja cualquier origen atacante con cabeceras de credenciales. Permitir `60mb` en el parser global de JSON permite a un atacante enviar payloads masivos a endpoints de consulta ligera, saturando la memoria Heap del proceso hasta provocar el cierre por *Out-Of-Memory (OOM Killer)* en el VPS.

● CRIT-05: Inyección Directa de Prompts y Sobrecarga Masiva de Contexto en J.A.R.V.I.S.

OWASP TOP 10 FOR LLM (LLM01/LLM04)

Ubicación: `services/jarvisService.js:332-374` | Severidad: MEDIA / ALTA (Costos & Latencia)

✦ **Impacto:** El catálogo completo (44+ posters con todos sus metadatos) se inyecta en el `systemInstruction` en cada turno de chat. Con un catálogo creciente, cada mensaje consumirá más de 20,000 tokens de entrada, elevando la latencia a más de 4 segundos y multiplicando los costos de cuota de API. Además, el input del cliente no cuenta con delimitación semántica estricta contra inyecciones de instrucciones.

4. 🚨 Áreas de Mejora (Nivel Amarillo — Hacia Grado Empresarial)

Dimensión	Estado Actual	Estado Objetivo (Grado Empresarial)	Prioridad
Almacenamiento de Medios	Archivos locales en /app/public/posters/uploads	Object Storage (S3 / Cloudflare R2) vía Presigned URLs	ALTA (12-Factor VI)
Cabeceras de Seguridad HTTP	Ausentes (solo compresión Gzip)	Middleware helmet (CSP, HSTS, X-Frame-Options)	ALTA (OWASP A05)
Rate Limiting	Map in-memory sin TTL ni desalojo	Token Bucket en Redis o Gateway (Traefik / Nginx)	MEDIA (Mem Leak)
Paginación de API Pública	getAllPosters() vuelca toda la base de datos	Paginación basada en cursores (cursor, take)	MEDIA (LCP / INP)
Validación de Esquemas	Validaciones manuales en controladores	Validación declarativa con Zod schemas	MEDIA (Data Quality)
Seguridad de Contenedores	Ejecución bajo usuario root en Docker	Usuario no privilegiado USER_node en Dockerfile	ALTA (Container Sec)
Observabilidad y Logs	console.log sin estructurar	Structured JSON Logging con pino + Request ID	BAJA (DevOps)

Detalle de Puntos Arquitectónicos Clave:

- **Violación del Factor VI (12-Factor App – Procesos sin Estado):** Al almacenar las imágenes en el disco local del VPS, no es posible levantar 2 réplicas del backend en Dokploy para balancear carga, ya que la réplica B no tendrá acceso a los archivos subidos en la réplica A.
- **Fuga de Memoria en Rate Limiting:** En middleware/rateLimit.js, el objeto new Map() acumula direcciones IP indefinidamente sin un mecanismo de recolección de basura o TTL, lo que genera un crecimiento lineal de memoria heap bajo tráfico masivo.

5. 🗺️ Roadmap Estratégico de Madurez (3 Fases Exactas)

🎯 PASO 1 (Inmediato — Próximas 24 a 48h): Blindaje Zero-Trust

SPRINT INMEDIATO

Cerrar vectores de ataque directo, vulnerabilidades OWASP y fugas de memoria sin alterar contratos de frontend.

- **Instalar y activar Helmet:** Proteger cabeceras HTTP (Content-Security-Policy, X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN).
- **Restringir CORS y Body Limits:** Limitar orígenes estrictamente a https://decovintage.online y reducir el límite global de JSON a 2mb.
- **Manejo Seguro de Secretos (Fail-Fast):** Abortar el arranque si faltan ADMIN_SECRET o ADMIN_PASSWORD; aplicar crypto.timingSafeEqual en la verificación de tokens.
- **Contenedor Non-Root:** Agregar la directiva USER_node en el Dockerfile.

 PASO 2 (Medio Plazo — Sprint 2): Purga de Split-Brain y Paginación

SPRINT DE CONSOLIDACIÓN

Consolidar una única fuente de verdad en PostgreSQL y liberar el Event Loop de operaciones síncronas.

- **Eliminación de catalogStore.json:** Migrar entidades Category y StoreSettings a modelos formales en schema.prisma y eliminar todos los métodos fs.*Sync.
- **Paginación Cursor-Based en Prisma:** Implementar parámetros cursor y take en /api/catalog/posters para optimizar la carga del catálogo y mejorar métricas Core Web Vitals (LCP/INP).
- **Validación con Zod:** Centralizar la validación de payloads de entrada en controladores.

 PASO 3 (Escala & Eficiencia — Sprint 3): Desacoplamiento de Medios y RAG en IA

SPRINT DE ESCALA

Habilitar el escalado horizontal multi-instancia y reducir el consumo de tokens en un 85%.

- **Object Storage (Cloudflare R2 / S3):** Subida de imágenes vía URLs prefirmadas (Presigned URLs) desacoplando el almacenamiento de medios del VPS.
- **Arquitectura RAG / Semantic Filtering en J.A.R.V.I.S.:** Remover el volcado estático del catálogo del system prompt; utilizar la herramienta explorar_catalogo con búsqueda contextual inyectando únicamente las 3 a 5 obras relevantes por turno conversacional.
- **Rate Limiting Distribuido:** Reemplazar el Map local por Redis para compartir contadores entre instancias del backend.

6. Calificación de Arquitectura y Veredicto

Dimensión Evaluada	Calificación	Estado	Dictamen Técnico
Integridad de Datos y Persistencia	8.5 / 10	 Sólido	Prisma garantiza consistencia relacional; pendiente eliminar el archivo JSON legacy.
Seguridad de Aplicación (AppSec)	4.0 / 10	 Crítico	Fallbacks de claves en claro, timing attacks y CORS permisivo requieren parche inmediato.
Resiliencia del Event Loop & I/O	5.5 / 10	 Regular	I/O síncrono en disco y body limits masivos comprometen la concurrencia.
Eficiencia del Motor de IA (J.A.R.V.I.S.)	6.0 / 10	 Funcional	Excelente cascada de fallback, pero sufre de sobrecarga de tokens y costo evitable.
Alineación con The 12-Factor App	5.0 / 10	 En Transición	Almacenamiento de imágenes local en disco bloquea el escalado horizontal.

Dictamen Staff: La arquitectura construida cuenta con cimientos modernos y viables. Ejecutando el **Paso 1 (Seguridad Zero-Trust)** y la **Purga del Split-Brain (Paso 2)**, el sistema alcanzará la madurez y resiliencia requeridas para operar a nivel corporativo de alta disponibilidad.